Mining Specifications for Agentic Frameworks

Anonymous Author(s)

***Abstract*—Agentic AI frameworks provide abstractions for coordinating language models, tools, and stateful workflows. However, their APIs impose behavioral constraints that are not always explicit in reference documentation and often emerge through developer-reported failures, fixes, and discussions. Mining these artifacts offers an opportunity to recover implicit specifications governing framework API usage. We introduce *MINT* (Mining, Inference, Normalization, and Traceability), an LLM-assisted approach for inferring behavioral API contracts from heterogeneous developer artifacts. MINT employs three independent LLMs to infer candidate contracts, normalizes overlapping interpretations into distinct behavioral obligations, and preserves traceability to supporting evidence. We evaluate MINT on 845 developer instances collected from Stack Overflow, GitHub, and Hugging Face. MINT infers 1,043 normalized contracts from 667 instances (78.93%). Two independent LLM reviewers jointly assess 98.18% of contracts as evidence-supported and 88.59% as linked to the appropriate APIs, and find the contract sets complete in 82.31% of instances that produce contracts. Among 797 classified contracts, 92.22% concern single-method behavior, environmental prerequisites, or API selection. We further extend an established API-contract taxonomy with seven categories, accounting for 76.04% of classified contracts. These findings demonstrate the potential of LLM-assisted specification mining for agentic frameworks while highlighting challenges in inference completeness and taxonomy adequacy.**

***Index Terms*—specification mining, large language model, mining software repositories, API usage contracts**

I. Introduction

Specification-driven development (SDD) relies on explicit specifications of expected behavior to guide software implementation and validation. For AI agents built using Large Language Model (LLM) orchestration frameworks, correct behavior may depend on the interactions among external tools, framework components, and execution state across multi-step workflows. Incorrect configuration or coordination of these components can result in crashes, incorrect outputs, ignored tool calls, or loss of conversational context [1]. Detecting and preventing such failures requires understanding the conditions under which framework components are expected to interact correctly. However, these usage obligations may remain implicit rather than being explicitly documented as API specifications. Software repositories and developer discussions provide collective evidence of such obligations: client implementations reveal recurring usage patterns, issue reports and associated fixes document failures and corrective changes [1], while pull-request discussions capture requirements and decisions underlying code changes [2]. Mining this collective knowledge offers an opportunity to infer behavioral contracts governing agentic framework usage, providing a foundation for specification-driven development, fault detection, and automated program repair (APR).

Prior work on specification mining has explored two complementary sources of evidence: (1) source code and execution traces, and (2) software revision histories, API documentation, and developer discussions. In the first category, static approaches use frequent pattern mining to extract implicit programming rules [3], control-dependence analysis to infer API preconditions from client code [4], and partial-order mining to identify API usage protocols [5]. Complementary dynamic approaches infer temporal rules, parameterized multi-object protocols, and data-dependent behavioral models from execution traces [6]–[8]. More recently, LLM-based approaches such as SpecGen, AutoReSpec, and SpecPylot have demonstrated the feasibility of generating formal specifications from individual programs and refining them using verification feedback [9]–[11]. In the second category, approaches mine programming rules from software revision histories [12], extract API constraints from reference documentation [13], and identify informal API contracts from developer discussions [14]. Although these studies establish the feasibility of inferring specifications from diverse software artifacts, behavioral contracts governing the use of agentic frameworks remain insufficiently characterized. In particular, it remains unclear how heterogeneous developer evidence can be used to infer and consolidate reusable framework-level contracts, what types of behavioral obligations emerge, and whether existing API-contract taxonomies adequately represent them. These gaps motivate a systematic investigation of evidence-grounded contract inference, normalization, and characterization for agentic frameworks.

Our key insight is that developer-reported failures, corrective changes, and usage discussions provide complementary evidence of implicit behavioral obligations governing agentic framework APIs. By examining this evidence across independent client applications, we can infer candidate contracts that capture framework-level requirements rather than application-specific behavior. In this paper, we use the terms *behavioral specifications* and *behavioral contracts* interchangeably to refer to requirements that govern the expected behavior and usage of framework APIs. Building on this insight, we introduce *MINT*, an LLM-assisted approach for inferring and characterizing behavioral API contracts through four complementary operations: *Mining* heterogeneous developer artifacts, *Inference* of candidate behavioral and temporal contracts using independent LLMs, *Normalization* of overlapping interpretations into distinct, evidence-grounded contracts, and *Traceability* between inferred contracts and their supporting developer evidence. The resulting contracts describe requirements governing API configuration, invocation order, state dependencies, and observable behavior. Rather than specifying the func-

tionality of individual client applications, *MINT* focuses on identifying reusable obligations associated with the framework APIs they employ.

We evaluate *MINT* on 845 developer instances collected from Stack Overflow, GitHub, and Hugging Face, examining three complementary dimensions: contract inference effectiveness, behavioral contract characterization, and taxonomy adequacy. Our evaluation employs three frontier LLMs from distinct model families for independent contract inference, dedicated models for taxonomy design and classification, and two independent LLMs for contract and taxonomy quality assessment. *MINT* infers 1,043 normalized behavioral contracts from 78.93% of instances, with inference rates of 83.51% on Stack Overflow, 68.42% on GitHub, and 71.67% on Hugging Face. The two reviewers jointly assess 98.18% of contracts as evidence-supported and 88.59% as associated with the appropriate APIs, while finding contract sets complete in 82.31% of instances. Among 797 classified contracts, 92.22% concern single-method behavior, environmental prerequisites, or API selection. We extend an established API-contract taxonomy [14] with seven categories that capture recurring behavioral obligations in agentic frameworks, accounting for 76.04% (606/797) of the classified contracts. These findings demonstrate the potential of evidence-grounded specification mining and taxonomy refinement for characterizing behavioral requirements in agentic frameworks.

Our main contributions are as follows:

- *Evidence-Grounded Specification Mining.* We introduce *MINT*, an LLM-assisted framework that combines independent contract inference, evidence-grounded normalization, and review to recover behavioral specifications from heterogeneous developer artifacts while preserving traceability.
- *Empirical Evaluation of Inferred Contracts.* We systematically evaluate the effectiveness of contract inference, evidence support, API association, and completeness, and characterize how behavioral obligations vary across developer sources and agentic frameworks.
- *API-Contract Taxonomy Refinement.* We extend an established API-contract taxonomy with seven new categories capturing recurring behavioral obligations in agentic frameworks, and investigate their prevalence, classification consistency, and representational gaps.
- *Traceable Contract Corpus.* We construct a corpus of 1,043 normalized behavioral contracts from 845 developer instances across Stack Overflow, GitHub, and Hugging Face, accompanied by source evidence, inference outputs, normalization decisions, reviewer assessments, and taxonomy assignments to support future research.

The remainder of this paper is organized as follows. Section II presents the motivating example, Section III describes *MINT*, Section IV reports the empirical evaluation, and Section V presents the human evaluation. Sections VI–IX discuss lessons learned, related work, threats to validity, and conclusions, respectively.

LangChain CharacterTextSplitter Produces Chunks Larger Than chunk_size

Developer evidence stackoverflow **# 76633836** GitHub **# 10410**

1. The developer configured chunk_size=6, expecting chunks of at most six characters, but received one 26-character chunk.
2. The developer reported that the documentation did not explain why chunk_size failed to enforce the expected limit.

```
from langchain.text_splitter import
CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=6, chunk_overlap=2
)
splitter.split_text("abcdefghijklmnopqrstuvwxyz")

# ['abcdefghijklmnopqrstuvwxyz']
```

Observed problem: CharacterTextSplitter.split_text returned the entire 26-character input as a single chunk despite chunk_size=6, because no matching separator existed.

LangChain **(Version 0.0.285)**

```
..
splits = _split_text_with_regex(
    text, separator, self._keep_separator
)
..
return self._merge_splits(splits, _separator)
```

Key evidence: split_text splits at configured separators; _merge_splits does not subdivide oversized individual segments.

Documentation gap: Documentation (V 0.0.285) does not explain that chunk_size is not a strict upper bound.

MINT

Multi-Agent Contract Inference

Contract Normalization

API `CharacterTextSplitter.split_text`

Contract 1: split_text splits text only at occurrences of its configured separator (default "\n\n"). When no matching separator exists, the entire text is returned as a single chunk, even if its length exceeds chunk_size.

Contract 2: chunk_size is a best-effort maximum, not a strict upper bound. When the smallest separator-delimited piece exceeds chunk_size, the method may return a chunk larger than the configured limit.

Multi-Agent Contract Review: Supported and Complete

Human Validated **Taxonomy:** Intra-argument Contract

Fig. 1: Inferring behavioral contracts for LangChain's *CharacterTextSplitter.split_text* API. *MINT* consolidates candidate contracts into two evidence-grounded behavioral contracts, followed by independent review and taxonomy classification.

II. Motivation

A. A Real-World Example

Figure 1 illustrates how behavioral contracts can be inferred from real-world developer discussions. A developer reported unexpected behavior in LangChain's *CharacterTextSplitter.split_text* API.[1] The developer configured *chunk_size=6* and *chunk_overlap=2*, expecting the input text to be divided into smaller chunks. However, splitting a 26-character string returned the entire string as a single chunk, exceeding the configured size.

Understanding this behavior requires examining evidence beyond the reported output. The historical LangChain documentation describes character-based splitting but does not explicitly explain why chunks may exceed the configured size. To investigate, we examined the documentation and implementation of LangChain v0.0.285, a version identified in an independent GitHub issue reporting the same phenomenon.[2] The Stack Overflow post does not specify the developer's installed version. The historical implementation shows that *split_text* divides the input using a configured separator, which defaults to two consecutive newline characters, and subsequently merges the resulting pieces. Since the merging procedure does not split individual pieces that exceed *chunk_size*, the configured size is not a strict upper bound. Instead, the resulting chunk sizes depend on the available splitting points.

Inferring behavioral contracts from such evidence introduces another challenge. Given the same developer discussion, three independent LLMs proposed five candidate contracts: one model produced a single contract, while the other two proposed two each. Although the candidates described the same API behavior, they differed in scope and granularity, with some combining separator-dependent splitting and chunk-size limitations into a single contract. Retaining all five candidates would introduce overlap in descriptions, whereas selecting a single candidate could omit relevant behavioral conditions.

To address this challenge, *MINT* employs evidence-grounded normalization, consolidating the five candidates into two behavioral contracts, as illustrated in Figure 1. The first specifies that when the input contains no matching separator (*Precondition*), *split_text* returns the entire input as a single chunk, even if its length exceeds *chunk_size* (*Postcondition*). The second specifies that when a separator-delimited piece exceeds *chunk_size* (*Precondition*), *split_text* may return a chunk larger than the configured size (*Postcondition*), indicating that *chunk_size* is not a strict upper bound. Two independent LLM reviewers assessed both contracts as supported by the evidence and correctly associated with the API, and judged the resulting set complete. Human review of this example agreed with these assessments. Both contracts were subsequently assigned to an existing *intra-argument contract* category by the automated taxonomy classifier.

[1] https://stackoverflow.com/questions/76633836/

[2] https://github.com/langchain-ai/langchain/issues/10410

B. Observations

We draw three observations from this example and the broader challenges of inferring and characterizing behavioral contracts from developer discussions.

1 *Behavioral Contracts Are Not Always Explicitly Documented.* The developer encountered behavior whose conditions were not clearly explained in the examined documentation. While the discussion reports the unexpected result, the implementation clarifies why it occurs. Such documentation gaps obscure the API's behavioral constraints and can lead developers to assume guarantees that the API does not provide. Developer discussions offer valuable evidence of these implicit constraints, although deriving well-scoped behavioral contracts from reported problems requires careful interpretation.

2 *Independent Inference Produces Different Contract Interpretations.* Although the three LLMs examined the same discussion, their inferred contracts differed in scope and granularity. Some captured individual behavioral conditions, while others combined multiple aspects into a single contract. Simply aggregating these candidates introduces redundancy, whereas selecting a single interpretation risks overlooking relevant conditions. Producing a coherent contract set, therefore, depends on distinguishing overlapping descriptions from genuinely distinct behavioral conditions.

3 *Behavioral Contract Types Remain Insufficiently Characterized.* Developer discussions reveal diverse behavioral constraints across framework APIs, but these remain scattered across individual artifacts. Without systematic characterization, it is difficult to determine which contract types recur, how frequently they occur, and whether established API-contract taxonomies adequately represent them. Their distribution and the coverage of existing classifications thus remain open empirical questions.

C. Key Ideas

From these observations, we derive three key ideas that guide the design of *MINT*.

1 *Inferring Behavioral Contracts from Developer Discussions.* Developer discussions often describe API behavior through reported problems, observed outcomes, and explanations rather than explicit specifications. Translating these descriptions into behavioral contracts makes implicit constraints explicit by identifying the conditions under which an API exhibits a particular behavior. Preserving the original evidence and API association enables each inferred contract to be traced back to the discussion that supports it.

2 *Reconciling Independent Contract Interpretations.* Independent inference captures alternative interpretations of the same developer-reported behavior, but the resulting contracts may overlap or differ in granularity. Evidence-grounded normalization reconciles these interpretations by consolidating redundant descriptions while retaining distinct behavioral conditions. Independent review then assesses whether the normalized contracts remain supported by evidence, are correctly associated with their APIs, and are collectively complete. This

Algorithm 1: Algorithm of *MINT*

Input: Developer records $\mathcal{D}$; generators $\mathcal{G} = \{G_1, G_2, G_3\}$; reviewers $\mathcal{R} = \{R_1, R_2\}$; seed taxonomy $\mathcal{T}_0$
Output: Specifications $\mathcal{S}$; reviews $\mathcal{Q}$; taxonomy $\mathcal{T}$; assignments $\mathcal{A}$; traceability $\mathcal{L}$

$\mathcal{S}, \mathcal{Q}, \mathcal{L} \leftarrow \emptyset$;
/* Specification inference and normalization */
foreach $d \in \mathcal{D}$ **do**
  $E_d \leftarrow \textsc{ExtractEvidence}(d)$;
  $\mathcal{C}_d \leftarrow \emptyset$;
  foreach $G_i \in \mathcal{G}$ **do**
    $C_i \leftarrow \textsc{Infer}(G_i, E_d)$;
    $\mathcal{C}_d \leftarrow \mathcal{C}_d \cup C_i$;
  $(\mathcal{S}_d, L_d, z_d) \leftarrow \textsc{Normalize}(E_d, \mathcal{C}_d)$;
  $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_d$;
  $\mathcal{L} \leftarrow \mathcal{L} \cup \{L_d\}$;
  $\textsc{RecordDecision}(d, z_d)$;
  // Independent specification review
  $Q_d \leftarrow \emptyset$;
  foreach $R_j \in \mathcal{R}$ **do**
    $q_j \leftarrow \textsc{Review}(R_j, E_d, \mathcal{S}_d)$;
    $Q_d \leftarrow Q_d \cup \{q_j\}$;
  $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{Q_d\}$;
/* Taxonomy classification and refinement */
$\mathcal{A}_0 \leftarrow \textsc{Classify}(\mathcal{S}, \mathcal{T}_0)$;
$\mathcal{U} \leftarrow \textsc{Unresolved}(\mathcal{A}_0)$;
$\mathcal{B} \leftarrow \textsc{SampleAssigned}(\mathcal{A}_0)$;
$\Delta \leftarrow \textsc{Propose}(\mathcal{T}_0, \mathcal{U}, \mathcal{B})$;
$\Delta' \leftarrow \textsc{Assess}(\Delta)$;
$\mathcal{T} \leftarrow \textsc{FinalizeTaxonomy}(\mathcal{T}_0, \Delta')$;
$\mathcal{A} \leftarrow \textsc{Reclassify}(\mathcal{S}, \mathcal{T})$;
return $\mathcal{S}, \mathcal{Q}, \mathcal{T}, \mathcal{A}, \mathcal{L}$;

separates contract inference, consolidation, and assessment into distinct stages.

3 *Taxonomy-Guided Characterization of Behavioral Contracts.* Organizing inferred contracts using established API-contract categories enables systematic analysis of their types and distributions across framework APIs. Examining contracts that cannot be reliably classified can reveal recurring behavioral patterns that existing categories may not capture. This provides an empirical basis for assessing taxonomy coverage and proposing extensions where supported by the evidence.

III. Methodology

To investigate the implicit behavioral specifications of agentic frameworks, we conduct an empirical study using *MINT*, combining real-world evidence mining, LLM-assisted specification inference and normalization, and qualitative and quantitative analyses of the resulting specifications.

A. Data Collection

To infer behavioral contracts from developer reports, we collected records from three sources: Stack Overflow, GitHub, and Hugging Face Forums.

Stack Overflow. Following established keyword-based mining procedures [1], [15], [16], we collected Stack Overflow discussions in three stages using the Stack Exchange API [17]. 1 We searched using keywords such as *LLM agent*, *LangChain*, *LlamaIndex*, *Semantic Kernel*, *CrewAI*, *AutoGen*, and *OpenAI*, retrieving 2,929 posts. 2 We manually screened the posts, excluding irrelevant discussions, duplicates, and those with insufficient problem descriptions or code, retaining 951 unique posts. 3 We selected questions with accepted answers and retrieved their corresponding answer texts. After excluding missing or empty answers, the final corpus comprised 576 question–answer pairs.

GitHub. Following the repository-mining procedure of Islam et al. [1], we collected GitHub issues and commits in three stages. 1 We searched GitHub Topics using seven agentic frameworks: *LangChain*, *LangChain.js*, *LangGraph*, *LlamaIndex*, *CrewAI*, *AutoGen*, and *Semantic Kernel*, identifying 1,891 repositories. We excluded framework-maintained repositories to focus on client applications. 2 We collected closed issues and commits containing the keyword *fix*, randomly sampling up to 100 issues and 100 commits per framework while ensuring nonoverlapping selections. We excluded irrelevant records and those originating from sample projects or course materials. 3 We retrieved issue descriptions and discussions, as well as commit messages, changed files, and patches. Repeated issue mappings were consolidated, while distinct changes within the same commit were retained separately. The final corpus comprised 209 developer records from 91 repositories, including 69 issue-based and 140 commit-based records.

Hugging Face. Following the forum-mining procedure of Islam et al. [1], we collected developer discussions in three stages. 1 We searched Hugging Face Forums using the seven agentic-framework keywords employed in our GitHub collection. 2 We removed duplicate and irrelevant discussions, retaining 105 records. 3 We selected 60 posts with accepted answers, preserving the reported problems and their associated solutions as evidence for contract inference.

B. Evidence-Grounded Contract Inference

Given the collected corpus of developer discussions $\mathcal{D}$, *MINT* constructs source-specific prompts that incorporate relevant evidence for behavioral contract inference. For Stack Overflow and Hugging Face, the evidence consists of the reported problem, relevant code, and accepted solution. GitHub issues include the issue description and discussion, while commits include the commit message, affected code, and associated changes. These artifacts expose behavioral requirements through observed failures, explanations, and corrective actions rather than explicit specifications. Since a reported problem or suggested fix does not necessarily establish a contract, *MINT* distinguishes developer assumptions and incidental implementation details from supported behavioral obligations. For each record $d \in \mathcal{D}$, three independent LLMs examine the same evidence and infer candidate behavioral contracts without exchanging their interpretations. Each model identifies distinct requirements governing API inputs, configurations, execution order, state dependencies, or observable outcomes. A candidate

contract is expressed as a self-contained natural-language requirement specifying the conditions under which a behavior is required or prohibited. The inference instructions emphasize preserving concrete API names, dependencies, alternative solutions, and version-specific constraints while avoiding unsupported generalizations. Each contract is grounded in the supplied evidence rather than in the model's external knowledge. Each model produces a structured inference outcome containing a decision, candidate contracts, and a supporting rationale. Every candidate records its behavioral requirement and the API operations it governs. The decision distinguishes three cases: *contracts_found*, when one or more behavioral obligations are supported; *no_contract*, when sufficient evidence establishes no actionable requirement; and *abstain*, when missing or ambiguous evidence prevents reliable inference. Rather than selecting one model's interpretation, *MINT* retains the independently inferred candidates for subsequent normalization.

C. Contract Normalization and Review

Independent contract inference may produce descriptions of the same API behavior that differ in scope, granularity, or the conditions they apply to. Aggregating these candidates introduces redundancy, while selecting one interpretation risks omitting distinct requirements. To address this, *MINT* normalizes inferred contracts against their original developer evidence. Given an artifact d and candidate contracts $\mathcal{C}_d$, the normalizer produces a consolidated set $\mathcal{S}_d$ by examining their behavioral meaning, conditions, and associated APIs. Rather than relying on textual similarity or majority agreement, it distinguishes equivalent, overlapping, and distinct obligations. Redundant descriptions are consolidated, while different behavioral conditions are preserved separately. The resulting contracts may generalize multiple descriptions into a coherent requirement when supported by the original evidence. This process reduces redundancy while preserving behavioral specificity and avoiding unsupported assumptions.

We assume accepted answers correctly resolve reported problems and use them as reference evidence for contract inference and review. For GitHub records, issue discussions, or corrective patches, provide corresponding evidence. To mitigate hallucinations and self-assessment bias, *MINT* employs two independent LLM reviewers uninvolved in inference or normalization. Each assesses whether normalized contracts are evidence-supported, associated with the appropriate APIs, and collectively capture relevant behavioral requirements. Their assessments and rationales identify unsupported claims, missing conditions, and disagreements.

D. Taxonomy-Guided Contract Characterization

To characterize inferred contracts, *MINT* adopts the API-contract taxonomy of Khairunnesa et al. [14] as its seed taxonomy $\mathcal{T}_0$ (Figure 2). Given normalized contracts $\mathcal{S}$, an LLM classifier assigns each to the most appropriate leaf based on its behavioral obligation rather than the reported bug or root cause. Assignments include rationales and supporting evidence. Contracts without suitable categories are marked *unclassified*, those matching multiple categories *ambiguous*, and those lacking sufficient evidence *insufficient information*.

Because agentic framework contracts may exceed the existing taxonomy's scope, unresolved assignments can reveal missing categories or unclear boundaries. *MINT* employs a separate taxonomy designer to examine these cases alongside representative classified examples, following a designer–classifier approach similar to [18]. The designer distinguishes taxonomy gaps from classification errors and proposes minimal extensions for recurring unsupported behavioral distinctions. Each proposal includes a definition, inclusion and exclusion criteria, distinctions from existing categories, and supporting evidence. Extensions undergo structural checks and require support from multiple independent developer discussions; original categories remain unchanged.

The classifier and designer operate iteratively, using unresolved assignments to refine the taxonomy. After incorporating eligible extensions, *MINT* reclassifies affected contracts while preserving resolved assignments where applicable. This produces an extended taxonomy $\mathcal{T}$ and contract-to-category assignments $\mathcal{A}$ that are traceable to their developer records. Comparing the two taxonomies supports analysis of contract distributions, category coverage, and recurring behavioral constraints within agentic frameworks.

E. Specification Traceability

Throughout the pipeline, *MINT* maintains traceability between developer records, inferred candidates, normalized contracts, reviewer assessments, and taxonomy assignments. Each normalized contract is linked to its originating developer artifact, supporting evidence, and associated APIs. *MINT* also preserves inference outputs, normalization decisions, and classification rationales to make the resulting specifications auditable. These links enable examination of how each behavioral obligation was inferred, assessment of its supporting evidence, and tracing disagreements or potential errors.

IV. Evaluation

We conduct an empirical study using *MINT* to systematically recover and characterize behavioral API contracts from real-world developer artifacts. Specifically, we investigate their recoverability, behavioral types, and coverage by existing API-contract taxonomies through the following research questions:

- **RQ1 (Contract Inference):** To what extent can behavioral API contracts be inferred from real-world developer artifacts?
- **RQ2 (Contract Characterization):** What types of behavioral API contracts emerge from developer-reported problems, and how are they distributed?
- **RQ3 (Taxonomy Adequacy and Refinement):** To what extent do existing API-contract taxonomies represent the inferred behavioral contracts, and what additional distinctions emerge from taxonomy refinement?

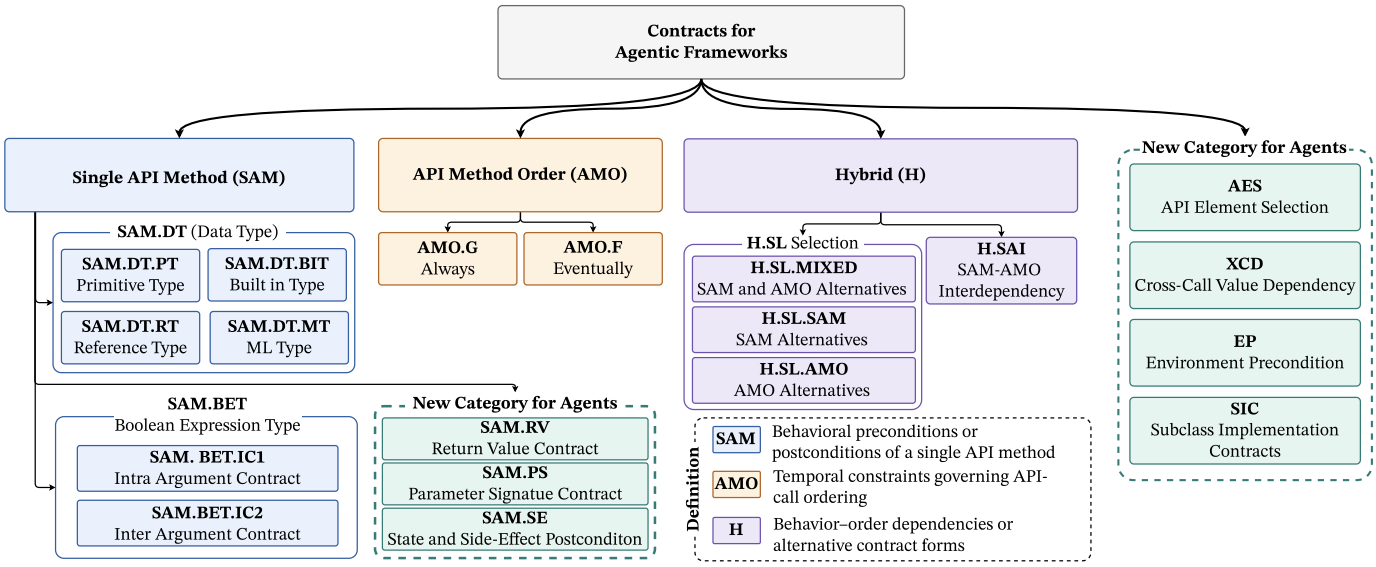


Fig. 2: Taxonomy of contracts for agentic frameworks. Green boxes highlight the seven new categories introduced by *MINT*.

TABLE I: Definitions and examples of the seven contract categories introduced by *MINT*.

Parent	Contract Category	Definition	Empirical Example
SAM	**SAM.RV** (Return Value)	A postcondition constraining the type, structure, content guarantees, or access form of a value returned, yielded, or streamed by an API method.	The completion text returned by `createCompletion` must be accessed through `response.data.choices[0].text`, rather than directly through `response.data` [SO].
	SAM.PS (Parameter Signature)	A precondition specifying permitted parameter names or keys, argument counts, or passing modes of an API method, independently of argument types and values.	Calls to `openai.completions.create` must specify the model using the `model` parameter rather than the deprecated `engine` parameter [SO].
	SAM.SE (State and Side-Effect Postcondition)	A postcondition specifying caller-observable effects of an API method beyond its return value, including changes to state, files, resources, or external services.	Following a successful `delete_collection` operation in AutoGen's pgvector integration, the corresponding database table must no longer exist [GH].
Root	**AES** (API Element Selection)	An obligation specifying which API method, class, constructor, endpoint, symbol, or module path must be used or avoided in a particular context.	In the reported LlamaIndex version, `ChatMessage` must be imported from `llama_index.core.llms` rather than `llama_index.llms` [SO].
	EP (Environment Precondition)	A precondition specifying environmental requirements for API usage, including package compatibility, runtime or platform constraints, configuration, and external-service state.	Using the legacy `openai.Audio.transcribe` interface requires Python version 3.7.1 or newer [SO].
	XCD (Cross-Call Value Dependency)	An obligation requiring a value supplied to one API call to match, derive from, or be forwarded from a value supplied to or produced by another call or pipeline stage.	The cancellation token supplied to `GroupChat.CallAsync` must be forwarded to `GetNextSpeakerAsync` and, when a speaker is selected, `GenerateReplyAsync` [GH].
	SIC (Subclass Implementation)	An obligation specifying required members, decorators, or state declarations in a user-defined implementation of an API-provided base class or interface.	A custom subclass of `MetadataFeatureExtractor` must implement the abstract `class_name()` method as a classmethod before it can be instantiated [SO].

A. Implementation

MINT is implemented in Python 3.12 using LangChain for LLM orchestration and OpenRouter [19] for model access. For contract inference, we use three independent LLMs: *Gemini 3.8 Flash* [20], *Claude Sonnet 5.5* [21], and *GPT-6.1 Sol* [22]. We employ *Claude Opus 5.5* [23] for evidence-grounded contract normalization. To mitigate hallucinations and self-assessment bias, two independent models, *Qwen3.8-Max* [24] and *DeepSeek-V4.1-Flash* [25], review the normalized contracts. For taxonomy-guided characterization, *Claude Sonnet 5.5* performs initial classification and reclassification, while *Claude Opus 5.5* serves as the taxonomy designer for iterative refinement. We set the reasoning effort to *low* for supported models and configure generation parameters separately for each stage, with a maximum output limit of 16k tokens for contract inference, normalization, and taxonomy refinement. Experiments were conducted on a server running Ubuntu 24.04.2 LTS, equipped with a 28-core Intel® Xeon® w7-3465X CPU, 256 GB RAM, and an NVIDIA RTX 6000 Ada GPU.

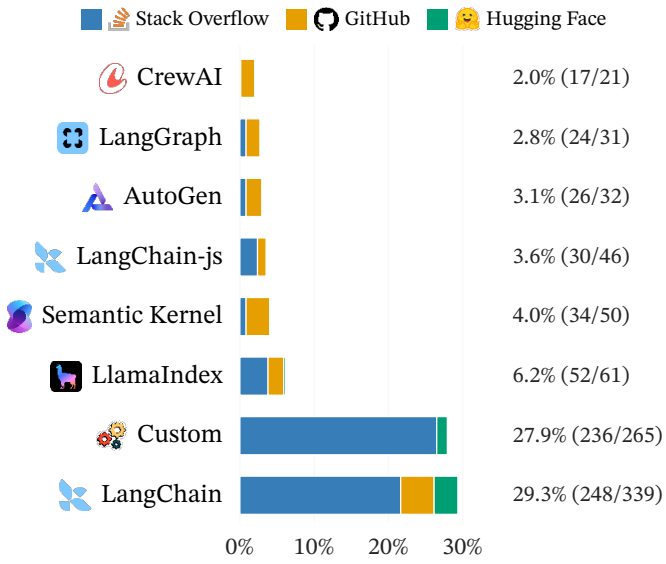


Fig. 3: Framework contributions to inferred contracts.

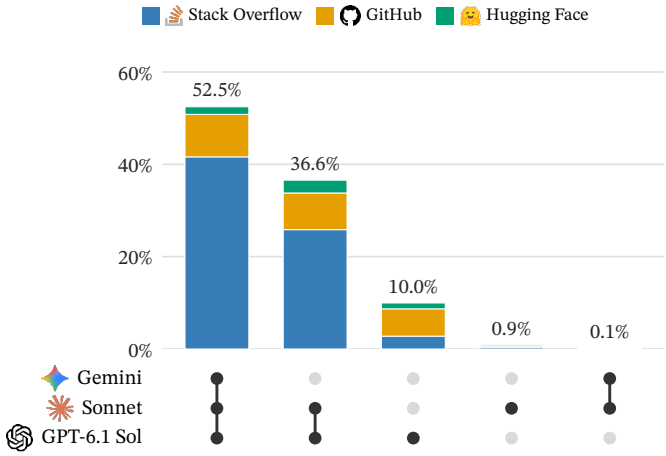


Fig. 4: Overlap in candidate contract inference.

B. RQ1: Contract Inference

Motivation Not all developer records provide sufficient evidence to infer behavioral API contracts. We therefore examine how frequently contracts can be recovered across different sources and assess whether they are supported by the evidence, associated with the correct APIs, and collectively capture the relevant behavioral requirements.

Method. We measure inference effectiveness as the proportion of developer instances yielding at least one normalized behavioral contract. We calculate inference rates separately for Stack Overflow, GitHub, and Hugging Face, reporting 95% confidence intervals. We use Pearson's chi-square (χ^2) statistic to examine differences across platforms, with significance determined through an artifact-cluster permutation test ($\alpha = 0.05$), and report Cramér's V to quantify the association. We also examine the overlap in candidate inference among the three LLM proposers. To evaluate contract quality, two independent LLM reviewers assess three dimensions: *evidence support*, whether a contract's behavioral claims are supported by the source evidence; *API association*, whether the contract is associated with the appropriate framework API; and *completeness*, whether the inferred contracts collectively capture the relevant behavioral expectations of an instance. Evidence support and API association are evaluated per contract, whereas completeness is evaluated per instance. We report approval rates requiring agreement from both reviewers and measure inter-reviewer agreement using observed agreement and Cohen's κ.

Results Inference Effectiveness. As shown in Fig. 3, LangChain and custom implementations contributed the largest shares of successfully inferred instances, accounting for 29.35% (248/845) and 27.93% (236/845), respectively. Across platforms, *MINT* inferred contracts from 83.51% (481/576) of Stack Overflow, 68.42% (143/209) of GitHub, and 71.67% (43/60) of Hugging Face instances. These differences were statistically significant ($\chi^2(2) = 23.04$, permutation $p < 0.001$), with a small association (Cramér's $V = 0.17$). Overall, *MINT* inferred 1,043 normalized behavioral contracts from 78.93% (667/845) of instances. Fig. 4 further demonstrates proposer complementarity: all three proposers generated candidates for 52.46% (416/793) of candidate-covered instances, while Sonnet and GPT-6.1 Sol jointly covered another 36.57% (290/793). GPT-6.1 Sol uniquely covered 9.96% (79/793), whereas Gemini contributed no exclusive coverage, indicating that proposer contributions were unequal.

Contract Quality. Both independent reviewers approved evidence support for 98.18% (1,024/1,043) and API association for 88.59% (924/1,043) of inferred contracts. The high evidence-support rate suggests that most contracts express behavioral expectations grounded in the corresponding developer evidence, while the lower API-association rate indicates that identifying the relevant framework APIs remains more challenging. Completeness reached 82.31% (549/667) among instances yielding normalized contracts, suggesting that even successfully inferred contracts may not collectively capture all behavioral expectations expressed in an instance. Raw inter-reviewer agreement was 98.50% for evidence support, 94.70% for API association, and 87.60% for completeness, with corresponding Cohen's κ values of 0.27, 0.67, and 0.69. Notably, evidence support exhibited high observed agreement but relatively low chance-corrected agreement, consistent with the predominance of positive judgments.

Answer to RQ1: *MINT* inferred contracts from 78.93% of developer instances, with 98.18% evidence support and 88.59% API association. However, completeness reached only 82.31% among successfully inferred instances, highlighting the gap between inferring grounded contracts and capturing complete behavioral expectations.

Together, these findings suggest that *MINT* generally infers evidence-grounded, API-relevant contracts, but establishing

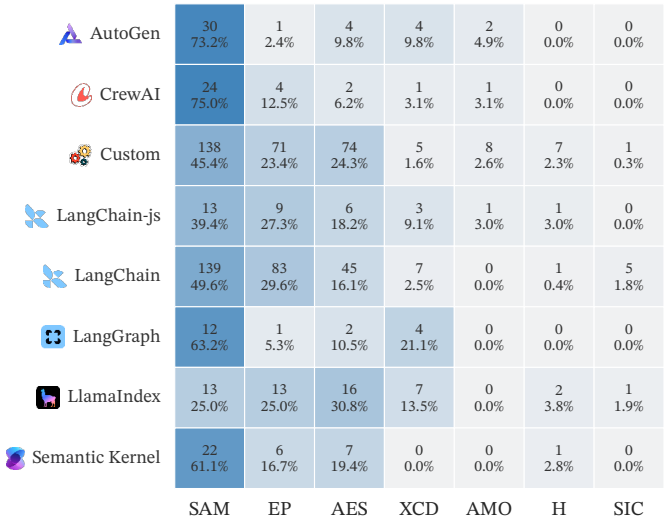


Fig. 5: Distribution of top-level behavioral contract categories across agentic frameworks. Each cell reports the number of classified contracts and their percentage within the corresponding framework. Darker shading indicates a higher proportion.

their completeness remains more difficult than establishing their support from the available evidence.

C. RQ2: Contract Characterization

Motivation. We investigate the prevalence of different behavioral contract types and how their distributions vary across datasets and agentic frameworks. Understanding these patterns can reveal recurring constraints in API usage and identify behavioral requirements that warrant greater attention in framework documentation and specifications.

Method. We characterize normalized contracts using taxonomy $\mathcal{T}$ (Figure 2). Claude Sonnet assigns each contract to a category or marks it as ambiguous or unclassified. DeepSeek and Qwen review the resolved assignments, accepting Sonnet's proposed category or suggesting an alternative. We measure assignment consistency using acceptance rates and pairwise Cohen's κ at parent and leaf levels, with instance-cluster bootstrap confidence intervals. Using Sonnet's assignments, we compare category frequencies across datasets and frameworks, using classified contracts within each group as the denominator. Statistical tests assess the presence of categories per developer instance, following the RQ1 procedure (§ IV-B) with a Holm correction for multiple comparisons.

Results. Contract-Type Prevalence. Claude Sonnet assigns 797 of 1,043 normalized contracts (76.41%) to resolved taxonomy categories. Among these 797 contracts, Single API Method (SAM) accounts for 49.06%, followed by Environment Precondition (EP; 23.59%) and API Element Selection (AES; 19.57%). Together, these categories represent 92.22%, indicating that inferred specifications primarily concern individual API behavior, execution prerequisites, and API selection rather than ordering constraints.

Fine-Grained Characterization. Among 391 SAM contracts, Return Value Contract (*SAM.RV*; 28.64%) and Intra Argument Contract (*SAM.BET.IC1*; 26.85%) predominate. Parameter Signature Contract (*SAM.PS*) and State and Side-Effect Postcondition (*SAM.SE*) each account for 14.32%. Together, these categories cover 84.14% of SAM contracts, emphasizing output, argument, and state requirements.

Framework-Level Variation. Figure 5 shows SAM dominating seven of eight framework groups; AES leads in LlamaIndex (30.77% of 52 contracts). Although uncommon, Cross-Call Value Dependency (*XCD*) accounts for 21.05% of 19 LangGraph contracts, compared with 2.50% of 280 LangChain contracts. Framework group is associated with instance-level XCD presence (Holm-adjusted $p = 0.02240$, Cramér's $V = 0.272$), without establishing pairwise differences.

Dataset Variation. Among classified contracts from Hugging Face ($n = 50$), GitHub ($n = 171$), and Stack Overflow ($n = 576$), EP represents 50% on Hugging Face versus 8.19% on GitHub; XCD represents 11.11% on GitHub versus 2.08% on Stack Overflow. Associations are significant for EP ($p_{\text{Holm}} = 0.00096$, $V = 0.204$) and XCD ($p_{\text{Holm}} = 0.00014$, $V = 0.298$), persisting in sensitivity analyses.

Taxonomy Assignment Consistency. DeepSeek and Qwen review 797 contracts from 560 developer instances. DeepSeek accepts 90.97% of Sonnet's leaf assignments and proposes 72 alternatives, including 37 parent-category changes. Qwen accepts 91.59% and proposes 67 alternatives, including 32 parent-category changes. Sonnet–DeepSeek agreement reaches Cohen's $\kappa = 0.931$ at the parent level and 0.895 at the leaf level; Sonnet–Qwen reaches $\kappa = 0.940$ and 0.902, respectively. DeepSeek–Qwen agreement reaches $\kappa = 0.942$ and 0.912. All three agree on 88.21% of leaf assignments, although Qwen proposes alternatives for 22 assignments accepted by DeepSeek. Since both reviewers see Sonnet's proposed labels, these results indicate consistency in anchored assignment rather than independent classification accuracy.

Together, these results reveal a concentration of common API obligations and broadly consistent taxonomy assignments, while less frequent constraints vary across datasets and frameworks. Unequal group sizes, reviewer anchoring, and unresolved assignments limit generalization.

Answer to RQ2: Among 797 classified contracts, 92.22% concern SAM, EP, or AES. DeepSeek and Qwen accept 90.97% and 91.59% of Sonnet's assignments, respectively, with 88.21% three-model leaf agreement. These findings suggest prioritizing common API-level obligations in specification mining and documentation while accounting for less frequent, framework-dependent constraints.

D. RQ3: Taxonomy Adequacy and Refinement

Motivation. Existing API-contract taxonomies may classify behavioral requirements without adequately capturing their meaning. We investigate representational gaps in agentic

framework contracts and assess the relevance of proposed extensions to the taxonomy.

Method. We compare historical assignments under the 12-leaf seed taxonomy $\mathcal{T}_0$ with those under the 19-leaf refined taxonomy $\mathcal{T}$ using matched Stack Overflow contracts. We examine reassignments, supporting evidence, the prevalence of seven introduced categories across datasets and frameworks, and the reviews by DeepSeek and Qwen of Sonnet's assignments. Following the RQ1 procedure (§ IV-B), we estimate uncertainty in coverage changes and unresolved rates by resampling developer instances and testing dataset differences while accounting for contracts from the same source artifact. Instances without contracts are excluded. Because classification settings changed between iterations, observed differences cannot be attributed solely to taxonomy refinement.

Results. Taxonomy Comparison. Across 716 matched Stack Overflow contracts, resolved coverage increased from 47.91% under the $\mathcal{T}_0$ to 80.45% under the $\mathcal{T}$, an increase of 32.54% (95% CI: 27.12–37.92). Of the 323 previously unresolved contracts subsequently classified, 312 were assigned to introduced categories, while 90 classified contracts became unresolved.

Value of Taxonomy Extensions. Across all three datasets, the seven introduced categories account for 76.04% of 797 classified contracts (606/797). Among these 606 assignments, DeepSeek retains 94.88%, and Qwen retains 93.73% of Sonnet's labels. Sonnet–Qwen agreement reaches 96.53% at the parent level and 93.73% at the leaf level. Agreement varies across categories: Qwen retains 83.93% of 56 state and side-effect assignments and 85.71% of seven subclass implementation assignments. These results support recurring behavioral distinctions, but because both reviewers were prompted with Sonnet's proposed labels, their agreement does not independently validate the taxonomy.

Remaining Gaps. Under the refined taxonomy, 23.59% of 1,043 contracts remain unresolved (246/1,043; 95% instance-cluster CI: 20.80–26.40%), comprising 153 ambiguous and 93 unclassified contracts. Instance-level unresolved rates vary across Stack Overflow (25.99%, 125/481), GitHub (44.06%, 63/143), and Hugging Face (23.26%, 10/43), with a significant omnibus association ($\chi^2(2) = 18.15$, permutation $p = 0.00016$, Cramér's $V = 0.165$).

Together, these findings suggest that taxonomy refinement exposes recurring behavioral distinctions supported by consistent reviews, while unresolved assignments and dataset-level variation highlight remaining representational limitations.

Answer to RQ3: Resolved classification increased by 32.54 percentage points across matched Stack Overflow contracts. The seven introduced categories account for 76.04% of classified contracts, with DeepSeek and Qwen retaining 94.88% and 93.73% of their assignments, respectively. However, 23.59% of contracts remain unresolved, with significant variation across datasets ($p = 0.00016$), indicating the need for further taxonomy refinement and independent validation.

V. Human Evaluation of Contracts and Taxonomy

Procedure. Our initial corpus contained 845 developer instances: 576 Stack Overflow posts, 209 GitHub records, and 60 Hugging Face discussions. Following prior work [1], [26], we used a 95% confidence level and a 5% target margin of error, and sampled 261 instances (174 Stack Overflow, 68 GitHub, and 19 Hugging Face) using dataset-specific, taxonomy-stratified sampling. The selected instances contained 455 normalized contracts. Two authors independently assessed evidence support, API association, taxonomy assignments, and instance-level completeness.

Normalized Contract Quality. Human reviewers agreed on evidence support for 95.60% (435/455; $\kappa = 0.395$), API association for 91.21% (415/455; $\kappa = 0.588$), and instance-level completeness for 90.04% (235/261; $\kappa = 0.538$). The lower κ for evidence support despite high raw agreement reflects predominantly positive ratings. These findings indicate consistent assessments, although agreement does not establish contract correctness or completeness.

Taxonomy Agreement. Reviewers assigned identical taxonomy labels to 82.64% (376/455) of contracts (Cohen's $\kappa = 0.804$), disagreeing on 79. Agreement on taxonomy-verification decisions reached 84.84% (386/455; $\kappa = 0.470$). These results indicate strong label agreement with the historical taxonomy, but do not validate the final refined taxonomy.

Ambiguous and Unresolved Assignments. We separately examined difficult taxonomy assignments using paired human annotations. Across 71 posts containing 86 contracts, reviewers agreed on 72.09% (62/86) of taxonomy labels (Cohen's $\kappa = 0.686$, 95% CI: 0.573–0.786). Agreement was lower than in the original review ($\kappa = 0.804$), suggesting greater difficulty distinguishing categories in these cases.

VI. Lessons Learned

1 *Evidence grounding does not guarantee contract completeness.* RQ1 (§ IV-B) shows that evidence-supported contracts may still omit relevant obligations. Researchers should evaluate completeness alongside grounding, while tool builders should identify missing requirements rather than treating supported contracts as complete specifications.

2 *Independent inference requires evidence-grounded normalization.* RQ1 (§ IV-B) also highlights that different LLMs may express overlapping obligations at different granularities. Tool builders should consolidate redundant interpretations while preserving distinct requirements, and researchers should investigate normalization methods that maintain evidence traceability without losing behavioral information.

3 *Behavioral obligations extend beyond individual API methods.* RQ2 (§ IV-C) reveals requirements concerning environmental prerequisites and API selection. Framework developers should document compatibility constraints, configurations, and API selection rules, while educators should emphasize these obligations in API-usage guidance.

4 *Contract distributions vary across frameworks and developer sources.* RQ2 (§ IV-C) also identifies variations in less frequent obligations, particularly cross-call dependencies. Tool

builders should incorporate framework-specific knowledge, while researchers should investigate how source characteristics influence the types of specifications that can be recovered.

5 *Taxonomy coverage does not establish semantic adequacy.* RQ3 (§ IV-D) reveals recurring distinctions alongside unresolved assignments. Researchers should independently validate taxonomy extensions, while tool builders should preserve uncertain classifications for further review rather than forcing category assignments.

VII. Related Work

Specification Mining and API Contracts. Prior work has mined API specifications from execution traces [6]–[8], [27]–[30], client code [3]–[5], revision histories [12], and documentation [13], [31], [32], as surveyed by Robillard et al. [33]. Recent studies also explore LLM-based specification generation and refinement for programs and APIs [9]–[11], [34]–[38]. Closely related to our work, Khairunnesa et al. [14] manually extracted informal contracts for ML APIs from Stack Overflow, which DL Contract [39] subsequently made checkable. Weng et al. [40] reported that 63% of expected constraints at LLM call sites in agentic systems were unmet. While existing approaches often infer specifications from observed behavior or usage patterns, *MINT* focuses on behavioral expectations revealed through developer-reported failures and fixes, consolidating them into traceable framework-level contracts.

Failures in LLM-Agent Software. Empirical studies have investigated failures in LLM agents [1], agentic frameworks [41], model–harness interactions [42], MCP software [43], [44], and coding agents [45]. These studies highlight configuration errors and component interactions as recurring challenges. For example, Zhang et al. [41] identified configuration-related triggering conditions in 92% of the framework bugs they analyzed. Other work develops techniques for agent diagnosis and repair [46]–[48], neural-model fault analysis [49], [50], and executable artifact reconstruction from developer reports [51], [52]. These efforts primarily characterize, reproduce, or address failures. In contrast, *MINT* identifies the behavioral requirements implicated in those failures and expresses them as explicit contracts that could support subsequent verification and testing.

LLM-Assisted Characterization of Developer Artifacts. Developer discussions provide valuable evidence of API information needs [53] and software requirements [2]. Recent approaches use LLMs to organize such evidence: TnT-LLM [54] generates taxonomies from text, while ATLAS [18] combines taxonomy design and classification with feedback-driven refinement. However, taxonomy construction still requires human oversight [55], classification outcomes can vary with prompting [56], and generated trace links may be unsupported [57]. Building on these insights, *MINT* starts with an established API-contract taxonomy [14], extends it to capture recurring behavioral requirements, and preserves links between inferred contracts and their supporting developer evidence.

VIII. Threats to Validity

Construct Validity. Developer-reported solutions, including accepted answers and merged fixes, may be incorrect or incomplete. Inferred contracts are version-dependent: constraints valid for older API releases may no longer apply after framework updates. We preserve source traceability to contextualize contracts. Two independent LLM reviewers assess evidence support, API association, and completeness; however, reviewers may share biases, and not all contracts were manually or execution-validated. Thus, evidence does not support behavioral correctness or current applicability.

Internal Validity. LLMs may hallucinate constraints, overlook implicit requirements, or misinterpret evidence. Normalization may merge distinct obligations or introduce unsupported generalizations. We mitigate these risks through three independent generators, structured normalization, and independent reviews. Nevertheless, model and prompt choices may influence inference outcomes. Taxonomy construction and classification also involve subjective decisions that may affect category boundaries and distributions.

External Validity. Our corpus comprises sampled Stack Overflow, GitHub, and Hugging Face artifacts. Mining and screening may introduce selection bias, while newly published or uncollected artifacts may reveal additional contracts. Consequently, neither our inferred contract collection nor the extended taxonomy is exhaustive. Findings characterize the studied evidence and should not be generalized to all frameworks, API versions, or development contexts.

IX. Conclusion

In this paper, we introduced *MINT*, an LLM-assisted approach for mining behavioral specifications from developer artifacts associated with agentic frameworks. By combining independent contract inference, evidence-grounded normalization, and traceability, *MINT* transforms developer evidence into explicit behavioral API contracts. Our evaluation across 845 developer instances yielded 1,043 normalized contracts, with 98.18% assessed as evidence-supported and 88.59% as associated with the appropriate APIs. We further characterized behavioral obligations across frameworks and developer sources, identifying a concentration of single-method behavior, environmental prerequisites, and API selection requirements. Extending an established API-contract taxonomy with seven categories revealed additional distinctions relevant to agentic frameworks. These findings demonstrate the potential of developer artifacts as sources of behavioral specifications while highlighting challenges in inference completeness and classification uncertainty. Future work will investigate stronger semantic validation, improved taxonomy refinement, and the translation of inferred contracts into executable specifications for automated verification and testing.

Data Availability

To support reproducibility, we provide a reproducible anonymized replication package containing the source code and the behavioral specifications at [58].

References

[1] N. Islam, R. S. Ayon, D. G. Thomas, S. Ahmed, and M. Wardat, "When agents fail: A comprehensive study of bugs in llm agents with automated labeling," 2026. [Online]. Available: https://arxiv.org/abs/2601.15232

[2] A. Oliveira, J. Correia, W. K. G. Assunção, J. A. Pereira, R. de Mello, D. Coutinho, C. Barbosa, P. Libório, and A. Garcia, "Understanding developers' discussions and perceptions on non-functional requirements: The case of the spring ecosystem," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: https://doi.org/10.1145/3643750

[3] Z. Li and Y. Zhou, "Pr-miner: automatically extracting implicit programming rules and detecting violations in large software code," *SIGSOFT Softw. Eng. Notes*, vol. 30, no. 5, p. 306–315, Sep. 2005. [Online]. Available: https://doi.org/10.1145/1095430.1081755

[4] H. A. Nguyen, R. Dyer, T. N. Nguyen, and H. Rajan, "Mining preconditions of apis in large-scale code corpus," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 166–177. [Online]. Available: https://doi.org/10.1145/2635868.2635924

[5] M. Acharya, T. Xie, J. Pei, and J. Xu, "Mining API patterns as partial orders from source code: From usage scenarios to specifications," in *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2007, pp. 25–34.

[6] D. Lo and S.-C. Khoo, "Mining patterns and rules for software specification discovery," *Proc. VLDB Endow.*, vol. 1, no. 2, p. 1609–1616, Aug. 2008. [Online]. Available: https://doi.org/10.14778/1454159.1454234

[7] C. Lee, F. Chen, and G. Roşu, "Mining parametric specifications," in *Proceedings of the 33rd International Conference on Software Engineering*, ser. ICSE '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 591–600. [Online]. Available: https://doi.org/10.1145/1985793.1985874

[8] D. Lorenzoli, L. Mariani, and M. Pezzè, "Automatic generation of software behavioral models," in *Proceedings of the 30th International Conference on Software Engineering*, ser. ICSE '08. New York, NY, USA: Association for Computing Machinery, 2008, p. 501–510. [Online]. Available: https://doi.org/10.1145/1368088.1368157

[9] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, "SpecGen: Automated generation of formal program specifications via large language models," in *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2025, pp. 16–28.

[10] R. S. Ayon and S. Ahmed, "AutoReSpec: A framework for generating specification using large language models," in *Proceedings of the 3rd ACM International Conference on AI Foundation Models and Software Engineering*, ser. FORGE '26, 2026, arXiv:2604.03758.

[11] ——, "SpecPylot: Python specification generation using large language models," in *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '26, 2026.

[12] V. B. Livshits and T. Zimmermann, "DynaMine: Finding common error patterns by mining software revision histories," in *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2005, pp. 296–305.

[13] X. Ren, X. Ye, Z. Xing, X. Xia, X. Xu, L. Zhu, and J. Sun, "API-misuse detection driven by fine-grained API-constraint knowledge graph," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 461–472.

[14] S. S. Khairunnesa, S. Ahmed, S. M. Imtiaz, H. Rajan, and G. T. Leavens, "What kinds of contracts do ml apis need?" *Empirical Softw. Engg.*, vol. 28, no. 6, Oct. 2023. [Online]. Available: https://doi.org/10.1007/s10664-023-10320-z

[15] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 510–520. [Online]. Available: https://doi.org/10.1145/3338906.3338955

[16] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1110–1121. [Online]. Available: https://doi.org/10.1145/3377811.3380395

[17] Stack Exchange, "Stack Exchange API v2.3," https://api.stackexchange.com/docs, accessed: 2026-10-22.

[18] J. Lu, M. Lyu, J. Wu, L. Yu, C. Liu, F. Zhang, L. Yang, C. Zuo, and Y. Liu, "ATLAS: Agentic taxonomy of large-scale software ecosystems," 2026. [Online]. Available: https://arxiv.org/abs/2606.21597

[19] OpenRouter, Inc., "OpenRouter: The unified interface for every model," 2026, accessed: October 14, 2026. [Online]. Available: https://openrouter.ai/

[20] Google DeepMind, "Gemini 3.8 flash model card," https://deepmind.google/models/model-cards/gemini-3-8-flash/, Sep. 2026, published September 2, 2026.

[21] Anthropic, "Introducing Claude Sonnet 5.5," Sep. 2026. [Online]. Available: https://www.anthropic.com/claude-sonnet-5-5

[22] OpenAI, "GPT-6.1 Sol: Model documentation," 2026. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-6.1-sol

[23] Anthropic, "Introducing Claude Opus 5.5," Sep. 2026. [Online]. Available: https://www.anthropic.com/claude-opus-5-5

[24] Alibaba Cloud, "Qwen3.8-Max: Model documentation," 2026. [Online]. Available: https://www.alibabacloud.com/help/en/model-studio/qwen3-8-max

[25] DeepSeek, "DeepSeek-V4.1-Flash: Smarter, faster, more efficient," Sep. 2026. [Online]. Available: https://api-docs.deepseek.com/news/news260910/

[26] R. S. Ayon, R. Fahmi, S. Biswas, and S. Ahmed, "ReqGenX: An Empirical Study of Atomic Decomposition, Artifact Regeneration, and Reconstruction for Legacy SRS Documents," in *20th International Symposium on Empirical Software Engineering and Measurement (ESEM 2026)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), R. Feldt, M. Paasivaara, D. Mendez, S. Wagner, and M. M. n. Barón, Eds., vol. 394. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 35:1–35:20. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESEM.2026.35

[27] G. Ammons, R. Bodík, and J. R. Larus, "Mining specifications," *SIGPLAN Not.*, vol. 37, no. 1, p. 4–16, Jan. 2002. [Online]. Available: https://doi.org/10.1145/565816.503275

[28] I. Krka, Y. Brun, and N. Medvidovic, "Automatic mining of specifications from invocation traces and method invariants," in *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014, pp. 178–189.

[29] A. Danese, T. Ghasempouri, and G. Pravadelli, "Automatic extraction of assertions from execution traces of behavioural models," in *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2015, pp. 67–72.

[30] M. Gabel and Z. Su, "Online inference and enforcement of temporal properties," in *Proceedings of the International Conference on Software Engineering*, 2010.

[31] H. Zhong, L. Zhang, T. Xie, and H. Mei, "Inferring resource specifications from natural language API documentation," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, 2009, pp. 307–318.

[32] X. Xu, F. Xie, C. Zhu, G. Bai, S. Khurshid, and Y. Li, "Identifying multi-parameter constraint errors in python data science library API documentation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, 2025.

[33] M. P. Robillard, E. Bodden, D. Kawrykow, M. Mezini, and T. Ratchford, "Automated API property inference techniques," *IEEE Transactions on Software Engineering*, vol. 39, no. 5, 2013.

[34] Z. Wang, T. Lin, M. Chen, H. Li, M. Yang, X. Yi, S. Qin, Y. Luo, X. Li, B. Gu, L. Lu, and J. Yin, "A tale of 1001 loc: Potential runtime error-guided specification synthesis for verifying large-scale programs," *Proc. ACM Program. Lang.*, vol. 10, no. OOPSLA1, Apr. 2026. [Online]. Available: https://doi.org/10.1145/3798268

[35] H. Ruan, Y. Zhang, and A. Roychoudhury, "SpecRover: Code intent extraction via LLMs," in *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2025.

[36] S. Deng, R. Huang, M. Zhang, C. Cui, D. Towey, and R. Wang, "LRASGen: LLM-based RESTful API specification generation," *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: https://arxiv.org/abs/2504.16833

[37] H. Huynh, Q.-T. Le, T. Nguyen, V. Nguyen, V. Nguyen, and T. N. Nguyen, "RBCTest: Leveraging LLMs to mine and verify oracles of

API response bodies for RESTful API testing," in *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: https://conf.researchr.org/details/icse-2026/icse-2026-research-track/247

[38] M. M. A. Naziri, S. Kim, F. Qin, M. d'Amorim, and S. Dutta, "Testing deep learning libraries via neurosymbolic constraint learning," in *Proceedings of the International Conference on Software Engineering*, 2026.

[39] S. Ahmed, S. M. Imtiaz, S. S. Khairunnesa, B. D. Cruz, and H. Rajan, "Design by contract for deep learning apis," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 94–106. [Online]. Available: https://doi.org/10.1145/3611643.3616247

[40] S. Weng, L. Wang, Y. Feng, M. Cheng, J. Liu, C. Shen, H. Ye, F. Sarro, and X. Xie, "When code becomes an LLM call: An empirical study of call-site contracts in agentic systems," 2026. [Online]. Available: https://arxiv.org/abs/2610.24545

[41] X. Zhang, H. Zhang, and S. H. Tan, "Understanding bugs in modern agentic frameworks: A study of symptoms, root causes, and triggering conditions," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '26, 2026.

[42] J. Chen, S. Chen, H. Zhu, J. Cao, J. Shen, and S.-C. Cheung, "Understanding agent-reactive bugs at the model-harness boundary: An empirical study of LLM agent issue reports," 2026. [Online]. Available: https://arxiv.org/abs/2607.15684

[43] M. Taraghi, M. M. Morovati, and F. Khomh, "Real faults in model context protocol (MCP) software: A comprehensive taxonomy," *ACM Transactions on Software Engineering and Methodology*, 2026, arXiv:2603.05637.

[44] S. He, C. Li, J. Liu, J. Liu, J. Keung, and X. Ma, "Understanding bugs in model context protocol development frameworks: An empirical study," *ACM Transactions on Software Engineering and Methodology*, 2026.

[45] A. A. Hasan and S. Biswas, "What breaks when LLMs code? characterizing operational safety failures of agentic code assistants," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '26, 2026.

[46] M. Chen, J. Wang, Z. Liu, Y. Wang, H. Zheng, and Q. Wang, "From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws," 2026. [Online]. Available: https://arxiv.org/abs/2606.06324

[47] R. Manke, M. Wardat, F. Khomh, and H. Rajan, "AgentInspect: Diagnosing behavioral failures in artificial intelligence agents," *Proc. ACM Softw. Eng.*, no. ISSTA, 2026.

[48] N. Islam, M. A. Raza, and M. Wardat, "SelfHeal: Empirical fix pattern analysis and bug repair in LLM agents," 2026. [Online]. Available: https://arxiv.org/abs/2604.17699

[49] S. Jahan, M. B. Shah, P. Mahbub, and M. M. Rahman, "Improved detection and diagnosis of faults in deep neural networks using hierarchical and explainable classification," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE '25, 2025.

[50] S. Jahan, S. Rajput, T. Sharma, and M. M. Rahman, "Hierarchical fault detection and diagnosis for transformer architectures," 2026. [Online]. Available: https://arxiv.org/abs/2604.28118

[51] R. S. Ayon, "Reconstructing executable programs from informal bug reports," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '26, 2026.

[52] J. Shin, N. S. Harzevili, R. Aleithan, H. Hemmati, and S. Wang, "Retrieval-augmented test generation: How far are we?" in *Proceedings of the International Conference on Software Engineering*, 2026. [Online]. Available: https://conf.researchr.org/details/icse-2026/icse-2026-research-track/121

[53] M. Liu, X. Peng, A. Marcus, S. Xing, C. Treude, and C. Zhao, "API-related developer information needs in stack overflow," *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4485–4500, 2022.

[54] M. Wan, T. Safavi, S. K. Jauhar, Y. Kim, S. Counts, J. Neville, S. Suri, C. Shah, R. W. White, L. Yang, R. Andersen, G. Buscher, D. Joshi, and N. Rangan, "TnT-LLM: Text mining at scale with large language models," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 5836–5847. [Online]. Available: https://arxiv.org/abs/2403.12173

[55] S. Nakashima, Y. Ishimoto, M. Kondo, T. Xiao, and Y. Kamei, "How well do LLMs generate taxonomies in the SE domain? a multi-perspective evaluation framework," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, 2026, pp. 3239–3251.

[56] A. Koyuncu, "Exploring fine-grained bug report categorization with large language models and prompt engineering: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 3, 2026.

[57] E. Alor, S. Khatoonabadi, and E. Shihab, "Evaluating the use of LLMs for documentation to code traceability," *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: https://arxiv.org/abs/2506.16440

[58] Anonymous Authors, "MINT: Mining specifications for agentic frameworks – replication package," https://anonymous.4open.science/r/mint-specification-mining/README.md, 2026, anonymous artifact repository for double-blind review.